

Discovering stable communities in massive temporal graphs

Jiaxing Fan

fan_jia_xin@163.com

School of Cyber Science and
Engineering, Zhengzhou University
Zhengzhou, Henan, China

Chenghan Zhang

chenghan0218@163.com

Henan Key of Laboratory of
Cyberspace Situation Awareness
Zhengzhou, Henan, China

Yan Liu*

ms.liuyan@foxmail.com

Henan Key of Laboratory of
Cyberspace Situation Awareness
Zhengzhou, Henan, China

ABSTRACT

Community search aims to retrieve the cohesive subgraph. However, few works address the problem in a temporal graph although the nodes and the edges in many real networks change over time. In this manuscript, we propose a novel (k, s, t) core which requires each node to have at least k stable neighbours to discover stable communities on temporal graphs. The model has a polynomial time complexity, achieving a balance between community cohesion and algorithm runtime. We compare the (k, s, t) core with (k, t) -active core and (k, h) -core theoretically and experimentally. Experiments on five datasets show the feasibility and effectiveness of our proposed algorithms.

CCS CONCEPTS

• **Information systems** → *Novelty in information retrieval.*

KEYWORDS

temporal graphs, community model, community search

ACM Reference Format:

Jiaxing Fan, Chenghan Zhang, and Yan Liu. 2024. Discovering stable communities in massive temporal graphs. In *2024 5th International Conference on Computing, Networks and Internet of Things (CNIOT 2024)*, May 24–26, 2024, Tokyo, Japan. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3670105.3670186>

1 INTRODUCTION

Communities are dense subgraphs that widely exist in many real-world networks, such as social networks, biological networks, collaborative networks, and communication networks. Therefore, community mining is a fundamental tool for analyzing network structure and organization and has gradually become a research hotspot in recent years. Currently, researchers have proposed various models to define communities, e.g., k -core[15], k -truss[5], k -clique[2], etc., all of which require nodes in the communities to have high connectivity from the viewpoint of structural constraints. In addition, extracting communities can be applied to lots of applications

in real-world scenarios, including collaborative group search[22], biological module discovery[10] and label recommendation[16].

However, Most of the works[1, 3, 6, 7, 9, 14, 17, 18, 20] only focus on structural cohesion, which works well on static graphs, but due to not considering temporal information, it can not apply to temporal graphs which each edge is associated with time. As shown in Figure 1, nodes $\{e, f, i, j\}$ form a 4-truss community, which has high cohesion in the structure. In fact, each node only interacts with each other up to two times, and the connections between them are weak, making the community fragile. Besides, these methods[11–13, 19] considering temporal information apply to specific scenarios, they cannot strike a balance between cohesion and computational efficiency.

To address the above limitations, we propose (k, s, t) core to extract communities on temporal graphs. The (k, s, t) core meets the following conditions: 1) Each node u has at least k stable neighbours. 2) There are at least s triangles composed of u and its every stable neighbour in the de-temporal graph appearing within t snapshots. 3) (k, s, t) core is the largest connected subgraph composed of t -temporal edges. This definition is reasonable. Guo et al. [8] clarify that in social networks, a triangle indicates that two friends have a common friend, and the more common friends two people have, the stronger their relationship becomes. Inspired by this, in a temporal graphs, if two nodes are contained in more triangles and these triangles appear at as many snapshots as possible, then the relationship between them is more stable. Besides, It does not require neighbors to co-occur in each snapshot which identifies the interaction behavior of neighbors at different period. In summary, the main contributions of this manuscript are the following:

(1) A new concept (k, s, t) core is proposed to model the community on a temporal graphs. This model ensures that nodes within the communities have a sufficient number of stable neighbours and retains "unstable edges" with sufficient interaction times to prevent subgraphs collapse caused by inaccurate evaluation of stability characteristics between nodes.

(2) We develop effective algorithms to achieve polynomial time complexity for (k, s, t) core decomposition, which can be effectively applied to massive temporal networks.

(3) We conduct extensive experiments on five real datasets, and the results show that our algorithm is feasible and effective. We have also compared it with the other two existing algorithms.

2 PROBLEM DEFINITION

2.1 Basic Concepts

Definition 1 k -core[15]. Given a static graph $G(V, E)$ and a positive integer k , The subgraph S is a k -core of graph G if and when the following conditions are satisfied: (i) For any node u in S , its degree

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions.acm.org.
CNIOT 2024, May 24–26, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1675-1/24/05
<https://doi.org/10.1145/3670105.3670186>

is not less than k . (ii) The subgraph S is the largest, and there is no subgraph $S' \supset S$ satisfies (i).

Definition 2 support. Given a static graph $G(V, E)$, the *support* of an edge $(u, v) \in E$, denoted by $\text{sup}(e, G)$, is defined as $|\{\Delta_{uvw} : u, v, w \in V\}|$, Δ_{uvw} represents a cycle with a length of 3.

Definition 3 k -truss[5]. Given a static graph $G(V, E)$ and a positive integer k , The subgraph S is a k -truss structure of graph G if and when the following conditions are met: (i) For any edge e in S , $\text{sup}(e, S) \geq k$. (ii) S is the largest, and there is no subgraph $S' \supset S$ satisfies (i).

Here we assume that $\mathcal{G}(\mathcal{V}, \mathcal{E})$ represents an undirected temporal graph, where $\mathcal{V}$ and $\mathcal{E}$ represent node set and temporal edge set respectively. Each temporal edge $e \in \mathcal{E}$ is a triple (u, v, t) , where u is the node in $\mathcal{V}$, t is the timestamp of the interaction between u and v . Given a temporal graph $\mathcal{G}$, we can extract a series of snapshots $G_1, G_2, \dots, G_T$, each snapshot is a static graph and all snapshots share the same node set. The de-temporal graph of $\mathcal{G}$, denoted by $G(V, E)$, meets the following conditions: (i) $V = \mathcal{V}$. (ii) $E = \{(u, v) | \exists (u, v, t) \in \mathcal{E}\}$. The de-temporal graph is a static graph that ignores edge timestamps. $\mathcal{S}$ is a temporal subgraph of $\mathcal{G}$, $\mathcal{V}(\mathcal{S})$ is the node set, $\mathcal{E}(\mathcal{S})$ is the edge set satisfying $\mathcal{E}(\mathcal{S}) = \{(u, v) \in \mathcal{E}, \forall u, v \in \mathcal{V}(\mathcal{S})\}$. We summarize the frequently used notations in Table 1.

Example 1 Figure 1 shows a temporal graph $\mathcal{G}$. It consists of five graph snapshots $\{G_1, G_2, G_3, G_4, G_5\}$. The left figure of Figure 2 shows the corresponding de-temporal graph of $\mathcal{G}$.

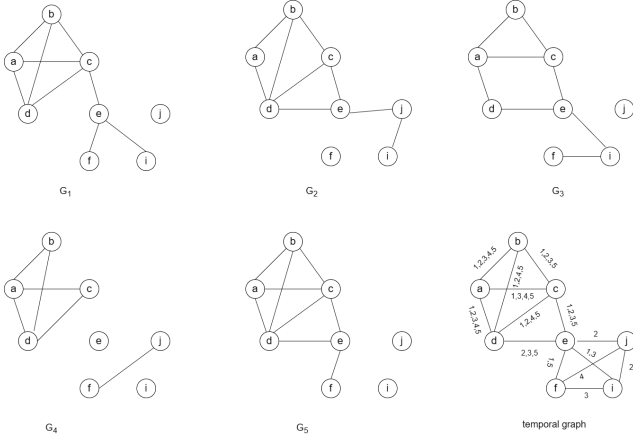


Figure 1: The temporal graph and corresponding snapshots.

Definition 4 t -temporal edge. Given a temporal graph $\mathcal{G}$ and its corresponding de-temporal graph $G(V, E)$, if edge $e = (u, v) \in E$ appears in at least t snapshots, then e is called a t -temporal edge.

Definition 5 stable neighbor. Given a temporal graph $\mathcal{G}$, its corresponding de-temporal graph $G(V, E)$, positive integer s and t , if at least s triangles where e is located appear in at least t snapshots, then nodes u and v are considered stable neighbour to each other.

Definition 6 stable neighbor set. Given a temporal graph $\mathcal{G}$ and a node u , stable neighbour set, denoted by $N_s(u, \mathcal{G})$, is the set of neighbours of u , each node in $N_s(u, \mathcal{G})$ is the stable neighbour of u .

Definition 7 (k, s, t) core. Given a temporal graph $\mathcal{G}$ and parameters k, s, t , (k, s, t) core, denoted by $S_{k,s,t}(\mathcal{G})$, is a subgraph of $\mathcal{G}$,

which satisfies the following condition: for any node u in $S_{k,s,t}(\mathcal{G})$, there are at least k stable neighbors.

Example 2. In the temporal graph $\mathcal{G}$, given $k=2, s=2, t=3$, as shown in figure 2, $(2,2,3)$ core is composed by a, b, c, d . In the de-temporal graph corresponding to these nodes, each node is in at least two triangles, and these triangles appear in at least 3 snapshots. For example, node a appears in triangle Δ_{abc} and Δ_{abd} , Δ_{abd} appears in the snapshots 1,2,4,5, Δ_{abc} appears in the snapshots 1,3,5.

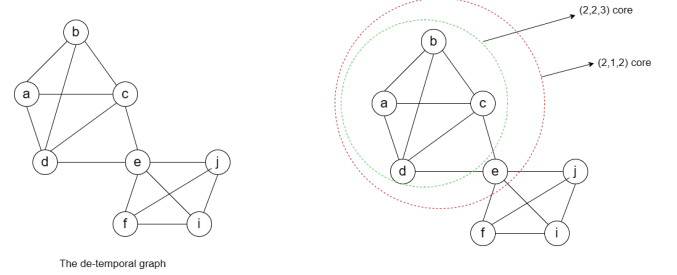


Figure 2: The de-temporal graph and examples for (k, s, t) core.

2.2 Properties of (k, s, t) core

Property 1 Given temporal graph $\mathcal{G}$, if $S_{k+i,s+j,t+m}(\mathcal{G}) \neq \emptyset$, there exists $S_{k+i,s+j,t+m}(\mathcal{G}) \subset S_{k,s,t}(\mathcal{G})$, $i, j, m = 0$ or 1.

Proof 1 This property is obvious. If a node u satisfies $S_{k+i,s+j,t+m}(\mathcal{G})$, it must also satisfy $S_{k,s,t}(\mathcal{G})$.

Property 2 Given temporal graph $\mathcal{G}$, the de-temporal graph corresponding to $S_{k,s,t}(\mathcal{G})$ is a $\max(k, s+1)$ -core.

Proof 2 According to definitions 5 and 7, for any node u , it has at least s common neighbours with a stable neighbour in the de-temporal graph, the node u has at least $s+1$ neighbours, so the de-temporal graph of $S_{k,s,t}(\mathcal{G})$ is a $\max(k, s+1)$ -core.

3 (K,S,T) CORE COMPUTATION

In this section, we develop an efficient algorithm for extracting (k, s, t) core. The properties of (k, s, t) core shows we can correctly compute the (k, s, t) core on a small k -core.

The main idea of computing (k, s, t) core is to remove unqualified vertices iteratively. Initialize a subgraph $S = \mathcal{G}$, for each node u in S , we check whether $|N_s(u, S)| \geq k$ is true, and then delete the nodes that do not satisfy the criteria. If there are no nodes that can be removed, the algorithm terminates. However, removing node u may cause some previously satisfied nodes to no longer meet the above criteria. Therefore, we need to iteratively check nodes, which is a very time-consuming process. According to property 2, we can use algorithm 2 to calculate $\max(k, s+1)$ -core from the de-temporal graph of S and remove nodes that are not in the $\max(k, s+1)$ -core, which greatly reduces the number of nodes that need to be checked. Pseudocode as shown in Algorithm 1.

The time complexity of Algorithm 1. The time complexity of performing $\max(k, s+1)$ -core decomposition on the de-temporal graph G is $O(m)$. Determining whether a node u satisfies property $|N_s(u, S)| < k$ has a time complexity of $O(2 \times T \times d_{\max}^2)$, where T is

Table 1: Summary of notations

Notations	Definition
$\mathcal{G}(\mathcal{V}, \mathcal{E})$	An undirected temporal graph
$G(V, E)$	The de-temporal graph of $\mathcal{G}$
$S, \mathcal{S}$	The subgraph of G and $\mathcal{G}$
e	An edge in G
u, v	The nodes of G and $\mathcal{G}$
$\text{sup}(e, G)$	The number of triangles edge e involves in G
n, m	The number of nodes and edges in G , respectively
$\text{Num}((u, v), \mathcal{G})$	The number of timestamps for interaction between nodes u and v
k, s, t	Threshold parameters
$\text{deg}(u, S), \text{deg}(u, \mathcal{S})$	The number of neighbors in S and $\mathcal{S}$
$N(u, S)$	The neighbor set of node u in S
$N_s(u, S)$	The stable neighbor set of node u in $\mathcal{S}$
$S_{k,s,t}(\mathcal{G})$	The (k, s, t) core of $\mathcal{G}$
$\mathcal{E}(u, S)$	The set of temporal edges where each edge is incident to u in S

Algorithm 1: ComputeKSTCore($\mathcal{G}, k, s, t$)

Input: temporal graph $\mathcal{G}$, integer k, s, t
Output: (k, s, t) core $S_{k,s,t}(\mathcal{G})$

- 1 Let G be the de-temporal graph of $\mathcal{G}$;
- 2 Let S be the $\max(k, s + 1)$ -core of G , $\mathcal{S}$ is the temporal graph of S ;
- 3 $Q = \emptyset$ //Initialize an empty queue ;
- 4 $\text{influSet} = \text{set}()$ //Store the neighbors of the deleted nodes;
- 5 $\text{vis} = \text{list}()$ //Mark nodes that have been deleted or will be deleted and initialize to False;
- 6 **for each node** $u \in \mathcal{S}$ **do**
- 7 **if** $|N_s(u, \mathcal{S})| < k$ **then**
- 8 $\text{vis}[u] = \text{True}$;
- 9 add u into Q ;
- 10 **while** True **do**
- 11 **while** $Q \neq \emptyset$ **do**
- 12 $u = Q.\text{pop}()$ // Stack a node;
- 13 **for each node** $v \in N(u, \mathcal{S})$ **do**
- 14 add v into influSet ;
- 15 $\mathcal{S} = \mathcal{S} \setminus \{u \cup \mathcal{E}(u, \mathcal{S})\}$
- 16 **for each node** v in influSet **do**
- 17 **if** $\text{vis}[v]$ is True **then**
- 18 continue;
- 19 **if** $|N_s(v, \mathcal{S})| < k$ and $\text{vis}[v]$ is False **then**
- 20 $\text{vis}[v] = \text{True}$;
- 21 add v into Q ;
- 22 **if** Q is Empty **then**
- 23 break;
- 24 return $\mathcal{S}$

the maximum temporal value and $d_{\max}$ is the maximum number of neighbors of a node in the temporal graph $\mathcal{G}$. So the time complexity

Algorithm 2: ComputeCore(G, k)

Input: static graph G , integer k
Output: $S_k(G)$

- 1 **while exists** $u \in G$ and $\text{deg}(u, G) < k$ **do**
- 2 $G = G \setminus \{u \cup E(u, G)\}$;
- 3 return G

of lines 6-9 is $o(2 \times T \times d_{\max}^2 \times n)$. In lines 10-21, for each removed node u , we need to check all its neighbor nodes, which has a time complexity of $o(\sum_{u \in \mathcal{G}} \text{deg}(u, G) \times 2 \times T \times d_{\max}^2) = o(4 \times m \times T \times d_{\max}^2)$. Therefore, the total time complexity of algorithm 1 is $o(m + n + 2 \times T \times d_{\max}^2 \times n + 4 \times m \times T \times d_{\max}^2)$. Since m is much larger than n , the time complexity is $o(m \times T \times d_{\max}^2)$.

The correctness of Algorithm 1. In Algorithm 1, whenever a node and its associated temporal edges are removed, the algorithm checks its neighbour node v to see whether it satisfies property $|N_s(v, \mathcal{S})| \geq k$. If not, we add v to the queue Q . At the end of the iteration, the remaining nodes satisfy property $|N_s(v, \mathcal{S})| \geq k$. Therefore, algorithm 1 can correctly find (k, s, t) core.

4 EXPERIMENTAL EVALUATION

In this section, we evaluate the effectiveness, performance and scalability of the algorithms proposed in this manuscript.

4.1 Experimental Setup

Datasets. We use five real-world temporal networks with different types and sizes to evaluate our algorithms, including social (Facebook, Wiki), email (Enron), citation (Arxiv), and collaboration (DBLP) networks, all of which can be found in <http://konect.cc/networks/>. Table 2 provides the details about the datasets.

Parameters and Environment. All algorithms are implemented in Python. The experiments run on a machine equipped with Intel CPU at 3.2 GHz and 32GM RAM. The default values for parameters k, s , and t are 3, 2, and 3, respectively.

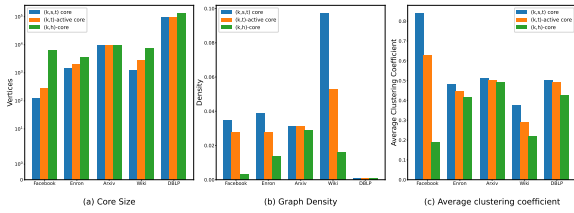
Table 2: Dataset statistics

Datasets	V	E	T	window size
Enron	87273	296844	47	one month
Facebook	46952	183412	52	one month
Arxiv	28093	3148447	92	one month
Wiki	1140148	2787965	76	one month
DBLP	1824701	8339314	49	one year

4.2 Effectiveness Evaluation

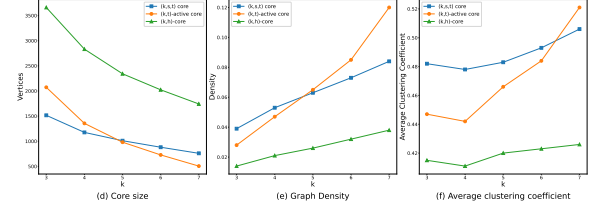
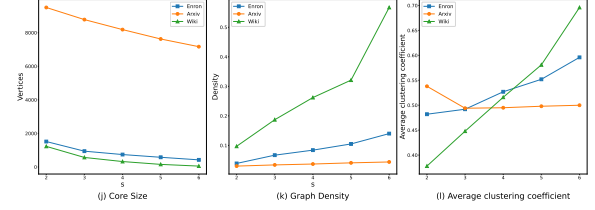
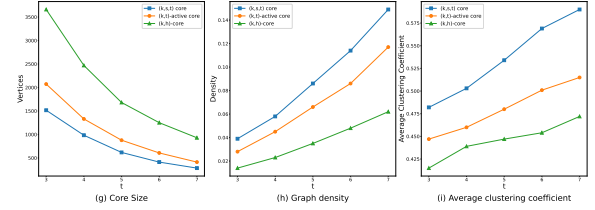
In this experiment, we choose (k, t) -active core[11] and (k, h) -core[19] as competitors. We select core size, density[4], and average clustering coefficient[21] in the de-temporal graph of the extracted core to evaluate the quality of the model.

Figure 3(a) - (c) shows the core size, density, and average clustering coefficients of the (k, s, t) core, (k, t) -active core, and (k, h) -core on five datasets. It is evident that compared to (k, t) -active core and (k, h) -core, the (k, s, t) core possesses the smallest core size, highest density, and greatest average clustering coefficient for all datasets. This observation is attributed to the (k, s, t) core taking into account the connection strength between nodes. Under identical conditions, it effectively eliminates nodes that lack close community ties, resulting in a more cohesive community. Figure 4(d) - (f) and figure 6(g) - (i) display the trends in the core size, density, and average clustering coefficient with respect to k and t , respectively. As k and t values increase, the core size diminishes, while the density and average clustering coefficient enhance across all methods. In the majority of instances, the (k, s, t) core exhibits superior quality. However, as k increases, the quality of the core obtained through the (k, t) -active core surpasses that of the (k, s, t) core. This is because the (k, t) -active core requires each node to have at least k co-neighbors across t snapshots. As k rises, its constraints become more stringent, allowing for the removal of additional nodes. Figure 5(j) - (l) displays the trend of three metrics with respect to s on Enron, Arxiv, and Wiki datasets. It can be clearly observed that as s increases, the core size in different datasets diminishes, while the density and average clustering coefficient enhance, leading to a more cohesive structure.

**Figure 3: Effectiveness of (k, s, t) core in different datasets.**

4.3 Efficiency Evaluation

In this section, we evaluate the efficiency of (k, s) -active core, (k, h) -core and (k, s, t) core. Table 3 and table4 depict the computing time of (k, h) -core, (k, s, t) core and (k, s, t) core for varying values of k and t , respectively. It is evident that On all datasets, (k, h) -core

**Figure 4: Effectiveness of (k, s, t) core with varying parameters k .****Figure 5: Effectiveness of (k, s, t) core with varying parameters s .****Figure 6: Effectiveness of (k, s, t) core with varying parameters t .**

has the shortest runtime due to its linear time complexity. With the increase of k , the computing time of (k, s) -active core increases sharply, and some cannot be calculated within the effective time. The parameter t has little effect on the computing time of (k, s) -active core. (k, s) -active core, (k, s, t) core has lower computation time in most cases with changes in parameters k and t .

Table 3: Efficiency of methods with varying parameters k

Datasets	Methods	2	3	4	5	6
Arxiv	(k, h)	3	3	3	3	3
	(k, t)	16	27	1584	-	-
	(k, s, t)	64	76	83	99	116
	(k, h)	10	9	8	9	9
Wiki	(k, t)	20	26	766	-	-
	(k, s, t)	57	53	50	44	42
	(k, h)	19	20	22	22	23
	(k, t)	58	60	143	1261	-
DBLP	(k, s, t)	57	63	47	51	40

Table 4: Efficiency of methods with varying parameters t/h

Datasets	Methods	2	3	4	5	6
Arxiv	(k, h)	3	3	3	3	3
	(k, t)	32	26	23	25	20
	(k, s, t)	48	74	88	87	95
Wiki	(k, h)	8	8	8	8	8
	(k, t)	43	24	18	16	15
	(k, s, t)	64	42	28	19	15
DBLP	(k, h)	16	20	23	23	24
	(k, t)	72	59	57	51	50
	(k, s, t)	70	57	42	34	37

5 CONCLUSION

In this manuscript, we propose a new temporal core model (k, s, t) core. The (k, s, t) core considers the temporal interaction information between nodes and achieves a balance between subgraph cohesion and computational efficiency. Besides, we propose an efficient algorithm to compute the (k, s, t) core. Extensive experiments validate the effectiveness of our model and the efficiency of our algorithm.

ACKNOWLEDGMENTS

This research was funded by Henan Province natural science Foundation project (222300420591), National Key Research and Development Program of China(2022YFB3102904). Conceptualization, Y.L. and J.F.; methodology, Y.L. and J.F.; validation, J.F.; formal analysis, J.F., L.T. and L.L.; data curation, J.F. and C.Z.; writing—original draft preparation, J.F. and Y.L.; writing—review and editing, C.Z. All authors have read and agreed to the published version of the manuscript.

REFERENCES

- [1] Esra Akbas and Peixiang Zhao. 2017. Truss-based community search: a truss-equivalence based indexing approach. *Proc. VLDB Endow.* 10, 11 (aug 2017), 1298–1309. <https://doi.org/10.14778/3137628.3137640>
- [2] Balabhaskar Balasundaram, Sergiy Butenko, and Illya V. Hicks. 2011. Clique Relaxations in Social Network Analysis: The Maximum k-Plex Problem. *Operations Research* 59, 1 (2011), 133–142. <https://doi.org/10.1287/opre.1100.0851>
- [3] Lijun Chang. 2019. Efficient Maximum Clique Computation over Large Sparse Graphs. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (Anchorage, AK, USA) (KDD '19)*. Association for Computing Machinery, New York, NY, USA, 529–538. <https://doi.org/10.1145/3292500.3330986>
- [4] Lingyang Chu, Yanyan Zhang, Yu Yang, Lanjun Wang, and Jian Pei. 2019. Online density bursting subgraph detection from temporal graphs. *Proc. VLDB Endow.* 12, 13 (sep 2019), 2353–2365. <https://doi.org/10.14778/3358701.3358704>
- [5] Jonathan Cohen. 2008. Trusses: Cohesive Subgraphs for Social Network Analysis. *national security agency technical report* (2008).
- [6] Alessio Conte, Tiziano De Matteis, Daniele De Sensi, Roberto Grossi, Andrea Marino, and Luca Versari. 2018. D2K: Scalable Community Detection in Massive Networks via Small-Diameter k-Plexes. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (London, United Kingdom) (KDD '18)*. Association for Computing Machinery, New York, NY, USA, 1272–1281. <https://doi.org/10.1145/3219819.3220093>
- [7] Alessandro Epasto, Silvio Lattanzi, and Mauro Sozio. 2015. Efficient Densest Subgraph Computation in Evolving Graphs (*WWW '15*). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 300–310. <https://doi.org/10.1145/2736277.2741638>
- [8] T Guo and C Zhao. 2016. A New Measurement of Link Prediction Based on Common Neighbors. *Journal of China University of Metrology* 27 (2016), 121–124.
- [9] Bay-Yuan Hsu, Chih-Ya Shen, and Ming-Yi Chang. 2020. WMEgo: Willingness Maximization for Ego Network Data Extraction in Online Social Networks. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (Virtual Event, Ireland) (CIKM '20)*. Association for Computing Machinery, New York, NY, USA, 515–524. <https://doi.org/10.1145/3340531.3411867>
- [10] Haiyan Hu, Xifeng Yan, Yu Huang, Jiawei Han, and Xianghong Jasmine Zhou. 2005. Mining coherent dense subgraphs across massive biological networks for functional discovery. *Bioinformatics* 21, suppl_1 (06 2005), i213–i221. <https://doi.org/10.1093/bioinformatics/bti1049>
- [11] Ling Li, Yuhai Zhao, Yuan Li, Fazal Wahab, and Zhengkui Wang. 2022. The most active community search in large temporal graphs. *Knowledge-Based Systems* 250 (2022), 109101. <https://doi.org/10.1016/j.knsys.2022.109101>
- [12] Rong-Hua Li, Jiao Su, Lu Qin, Jeffrey Xu Yu, and Qiangqiang Dai. 2018. Persistent Community Search in Temporal Networks. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, 797–808. <https://doi.org/10.1109/ICDE.2018.00077>
- [13] Hongchao Qin, Rong-Hua Li, Guoren Wang, Lu Qin, Yurong Cheng, and Ye Yuan. 2019. Mining Periodic Cliques in Temporal Networks. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, 1130–1141. <https://doi.org/10.1109/ICDE.2019.00104>
- [14] Seyed-Vahid Sanei-Mehri, Apurba Das, and Srikanta Tirthapura. 2018. Enumerating Top-k Quasi-Cliques. In *2018 IEEE International Conference on Big Data (Big Data)*, 1107–1112. <https://doi.org/10.1109/BigData.2018.8622352>
- [15] Ahmet Erdem Sariyüce, Buğra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Ümit V. Çatalyürek. 2013. Streaming algorithms for k-core decomposition. *Proc. VLDB Endow.* 6, 6 (apr 2013), 433–444. <https://doi.org/10.14778/2536336.2536344>
- [16] Mauro Sozio and Aristides Gionis. 2010. The community-search problem and how to plan a successful cocktail party. Association for Computing Machinery, New York, NY, USA, 939–948. <https://doi.org/10.1145/1835804.1835923>
- [17] Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, Lu Qin, and Yuting Zhang. 2021. Efficient and Effective Community Search on Large-scale Bipartite Graphs. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, 85–96. <https://doi.org/10.1109/ICDE51399.2021.00015>
- [18] Dong Wen, Lu Qin, Ying Zhang, Lijun Chang, and Ling Chen. 2019. Enumerating k-Vertex Connected Components in Large Graphs. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, 52–63. <https://doi.org/10.1109/ICDE.2019.00014>
- [19] Huanhuan Wu, James Cheng, Yi Lu, Yiping Ke, Yuzhen Huang, Da Yan, and Hejun Wu. 2015. Core decomposition in large temporal graphs. In *2015 IEEE International Conference on Big Data (Big Data)*, 649–658. <https://doi.org/10.1109/BigData.2015.7363809>
- [20] Bohua Yang, Dong Wen, Lu Qin, Ying Zhang, Lijun Chang, and Rong-Hua Li. 2019. Index-Based Optimal Algorithm for Computing K-Cores in Large Uncertain Graphs. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, 64–75. <https://doi.org/10.1109/ICDE.2019.00015>
- [21] Jaewon Yang and Jure Leskovec. 2012. Defining and evaluating network communities based on ground-truth. In *Proceedings of the ACM SIGKDD Workshop on Mining Data Semantics (Beijing, China) (MDS '12)*. Association for Computing Machinery, New York, NY, USA, Article 3, 8 pages. <https://doi.org/10.1145/2350190.2350193>
- [22] Dong Zheng, Jianquan Liu, Rong-Hua Li, Cigdem Aslay, Yi-Cheng Chen, and Xin Huang. 2017. Querying Intimate-Core Groups in Weighted Graphs. In *2017 IEEE 11th International Conference on Semantic Computing (ICSC)*, 156–163. <https://doi.org/10.1109/ICSC.2017.80>